ChuvaRota Visão: detecção do nível de água em ônibus alagados com YOLOv8

Global Solution 2026.2 · Applied Computer Vision · Engenharia de Software · 4º ano

Integrantes

Vinicius Barros, RM 97824

Resumo

Este trabalho apresenta o treinamento customizado de uma arquitetura de visão computacional, o YOLOv8, para detectar ônibus em ruas alagadas e estimar até onde a água chega, em quatro níveis: 0 a 20 cm, 20 a 45 cm, 45 a 100 cm e mais de 100 cm. O modelo faz parte do ChuvaRota, produto que avisa o cidadão de São Paulo sobre o risco de chuva: a previsão diz que vai chover, e a imagem mostra o quanto de água já está na rua. Foram usadas 1.008 imagens públicas do Zenodo, com licença CC BY 4.0, divididas em 810 para treino, 90 para validação e 108 para teste, que o modelo nunca viu. O treino partiu dos pesos do COCO, durou 60 épocas, com imagens de 640 pixels, e levou 33 minutos em uma GPU de notebook. No conjunto de teste, o modelo treinado detectou 71% dos ônibus alagados, contra 50% do YOLOv8n original, e acertou o nível exato da água em 76% dos casos, com erro de no máximo um nível em 98%. O mAP50 foi 0,59 e a inferência leva cerca de 13 milissegundos por imagem, o bastante para tempo real. Como as fotos vêm de redes sociais, sobretudo de cidades chinesas, o desempenho em São Paulo pode ser pior. Foram entregues o código, os pesos, uma API que recebe uma foto e devolve o alerta sem gravá-la, e um vídeo de demonstração. O trabalho mostra que um modelo pequeno já traz informação útil para a segurança no trânsito em alagamentos.

1. Introdução

Em um alagamento, o risco para quem se desloca depende de quanta água já está na rua, e a previsão do tempo não mostra isso. A orientação oficial da Defesa Civil é clara: nunca atravessar pontes, ruas ou avenidas alagadas, mesmo de carro, moto ou bicicleta, porque a força da água pode arrastar o veículo. Mas o cidadão só consegue julgar o perigo olhando para a rua, e muitas vezes subestima a água.

A visão computacional pode fazer essa leitura a partir de uma foto. Usei um ônibus como régua: por ser grande e ter altura conhecida, o quanto dele está submerso indica a profundidade. O resultado complementa o resto do ChuvaRota (previsão de chuva por região, alertas automáticos e assistente de orientações) com a imagem do que está acontecendo agora.

2. Objetivo

Treinar um modelo YOLOv8 que detecte ônibus em ruas alagadas e estime o nível da água em quatro faixas, medir seu desempenho em imagens de teste inéditas e disponibilizar o resultado como alerta de segurança para o cidadão no ChuvaRota, por meio de uma API simples, sem guardar as fotos.

3. Ferramentas

Ferramenta	Uso e justificativa
YOLOv8n (Ultralytics 8.4)	Arquitetura de detecção de objetos em um só estágio. A versão "n" tem 3,1 milhões de parâmetros e 8,6 GFLOPs: é pequena e rápida, adequada a poucas imagens e a tempo real.
Transferência de aprendizado (pesos do COCO)	Com só 810 imagens de treino, partir de um modelo que já sabe achar ônibus é muito melhor do que treinar do zero.
PyTorch 2.x com GPU Apple (MPS)	Motor de treino; a GPU integrada do Mac permitiu treinar sem servidor.

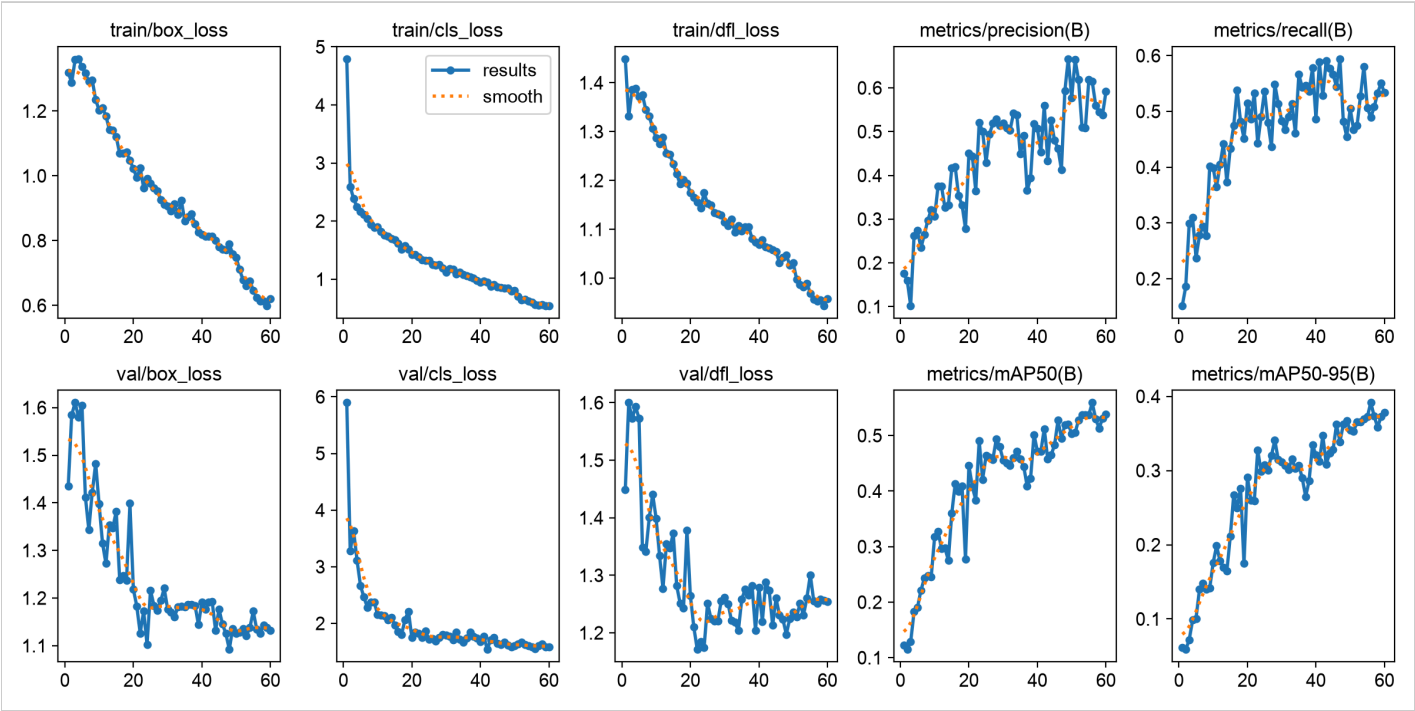
OpenCV	Leitura de imagens e vídeos, desenho das caixas e criação dos vídeos de teste.
FastAPI e pytest	API `POST /analisar` e seus testes.
Python 3.12, ffmpeg	Scripts e montagem do vídeo de demonstração.

4. Metodologia

Passo 1: dados. Procurei bases abertas que eu pudesse baixar sem criar conta. Os conjuntos do Roboflow exigem chave de API e o "Flood Dataset-1" do Mendeley tinha só 300 imagens sem anotação. Usei o conjunto do Zenodo (CC BY 4.0): 1.008 imagens de ônibus em alagamento, coletadas de redes sociais e anotadas em formato YOLO, com 4 classes de nível de água: 0 a 20 cm, 20 a 45 cm, 45 a 100 cm e mais de 100 cm.

Passo 2: preparação. O script `preparar_dados.py` sorteou (semente 42) 90 das 900 imagens de treino para validação e manteve 810 para treino. As 108 imagens de validação originais viraram o **teste**, sem nenhum uso no treino ou na escolha do modelo.

Passo 3: treino. `treinar.py`: YOLOv8n com pesos do COCO, 60 épocas, imagens de 640 px, lote de 16, semente 42, parada antecipada com paciência 15 e aumento de dados padrão (mosaico, espelhamento, variação de cor). Levou 33 minutos e o melhor ponto foi a época 56 (mAP50 de validação 0,56).



Curvas do treino: perdas caem e as métricas de validação sobem até o fim.

Passo 4: avaliação. `avaliar.py` mede, só no teste: mAP50 e mAP50-95, precisão e recall por classe; a capacidade de achar o ônibus (caixa com IoU de pelo menos 0,5, ignorando o nível), comparada com o YOLOv8n original do COCO; o acerto do nível; e a matriz de confusão.

Passo 5: uso. `detectar.py` detecta em imagem, vídeo ou webcam e escreve o alerta; `api.py` expõe o modelo por `POST /analisar`, com validação do arquivo (imagem de até 5 MB) e sem gravar a foto.

5. Resultados

5.1 Métricas no conjunto de teste (108 imagens, 168 ônibus)

mAP50	mAP50-95	Precisão	Recall
0,59	0,41	0,64	0,54

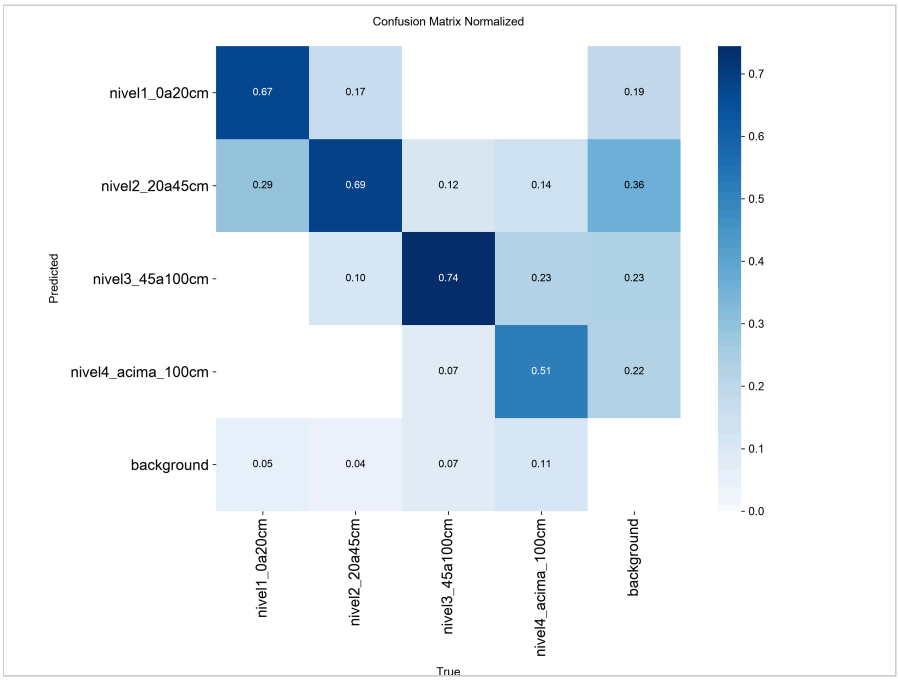
Classe	AP50	AP50-95
nivel1_0a20cm	0,71	0,49
nivel2_20a45cm	0,55	0,38
nivel3_45a100cm	0,62	0,41
nivel4_acima_100cm	0,49	0,34

Essas métricas exigem que a caixa **e** o nível estejam certos, por isso são mais baixas que as da tabela seguinte, que só avalia se o ônibus foi encontrado.

5.2 Modelo treinado contra o YOLOv8n do COCO

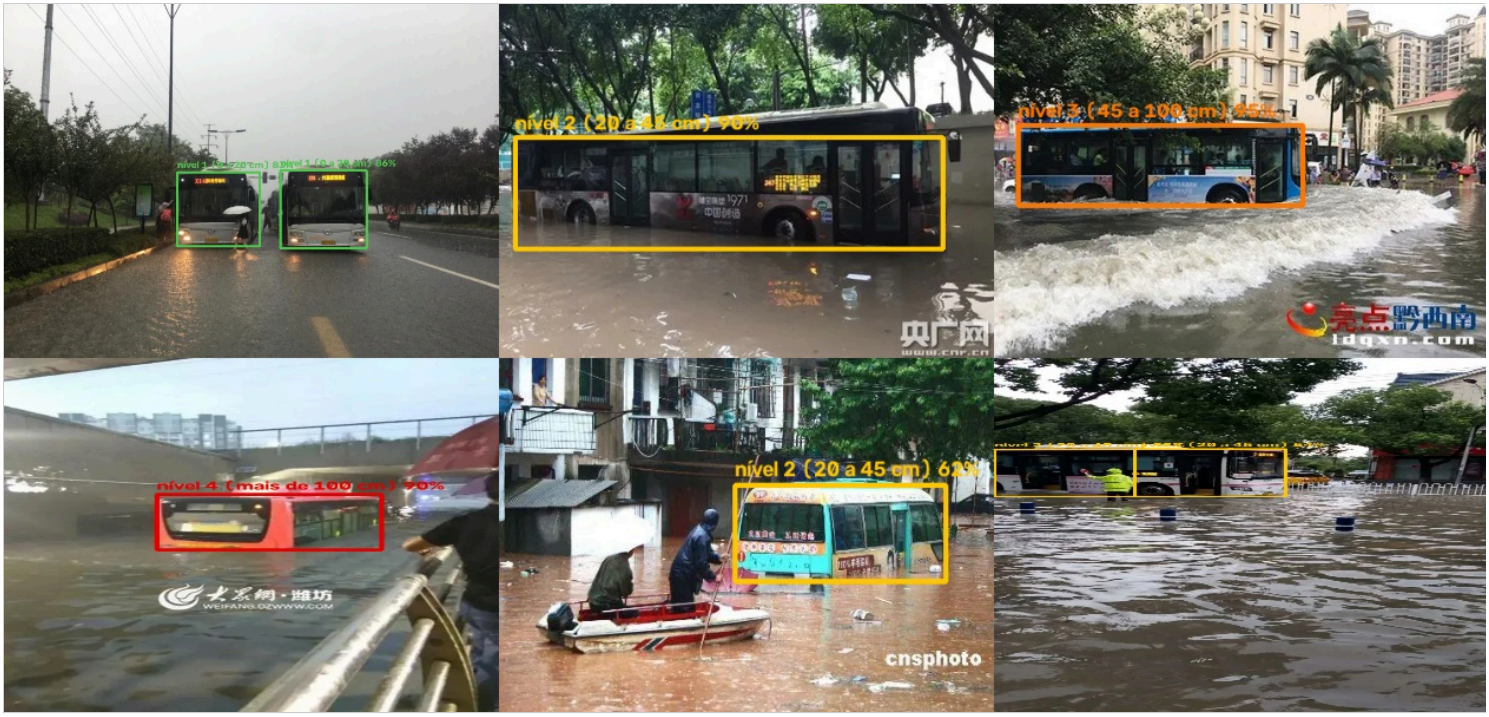
	Modelo treinado	YOLOv8n do COCO
Recall (acha o ônibus alagado)	71%	50%
Precisão	75%	72%
F1	0,73	0,59
Nível da água exato	76% (91 de 120)	não estima
Nível com erro de no máximo 1	98%	não estima

O treino melhorou a detecção dos ônibus alagados e deu ao modelo algo que ele não tinha: o nível da água. Não investiguei por que o modelo original perde tantos ônibus; uma hipótese é a água cobrindo parte do veículo.



Matriz de confusão normalizada no teste: os erros se concentram nos níveis vizinhos.

5.3 Exemplos



Quatro acertos (um de cada nível) e dois erros: no 5º (embaixo, no centro) um ônibus com mais de 100 cm foi dado como nível 2, com 62% de confiança; no 6º o nível está certo, mas a caixa não cobre bem o ônibus.

5.4 Velocidade

A inferência leva cerca de 13 ms por imagem no teste (GPU do Mac). No script de vídeo, com leitura, desenho e gravação, o ritmo ficou em torno de 15 quadros por segundo, suficiente para tempo real em câmera de rua.

6. Limitações

- As fotos vêm de redes sociais, principalmente de cidades chinesas: ônibus, ruas e câmeras diferentes das de São Paulo, então o desempenho aqui pode ser pior e precisa ser medido com fotos locais.
- O modelo só conhece ônibus; carros, motos e pessoas ficam de fora.
- O nível 4 (mais de 100 cm) é o mais fraco (AP50 de 0,49): há poucos exemplos e o ônibus quase desaparece na água.
- Exigindo que todos os ônibus de uma foto estejam com caixa e nível certos (confiança 0,4), 50 das 108 fotos do teste (46%) ficam totalmente certas.
- O teste tem só 108 imagens, então as porcentagens têm margem de erro grande.

7. Conclusão

Um YOLOv8 pequeno, ajustado em 33 minutos com 810 imagens, passou a achar ônibus alagados melhor que o modelo original (71% contra 50%) e a estimar a água com erro de no máximo um nível em 98% dos casos. Para o ChuvaRota, isso já permite transformar uma foto em um alerta de "não atravesse". Os próximos passos são treinar com fotos de São Paulo, incluir outros veículos e pessoas, e juntar o resultado ao risco de chuva por região.

8. Links

Repositório: https://github.com/ViniciusBarrosFiap/chuvarota (pasta `visao/`)
Vídeo: https://github.com/ViniciusBarrosFiap/chuvarota/blob/main/visao/amostras/video_visao.mp4
Dados: QIU, Yanbin (2025). *Automated urban flood level detection based on flooded bus dataset wading image using YOLOv8*. Zenodo, doi.org/10.5281/zenodo.17151262, licença CC BY 4.0. Usei só as imagens e anotações originais ("Original.zip"), sem alterá-las.